

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
ASSESSMENT TOOL 1 – RAG-BASED MINI ASSIGNMENT (DOCUMENT QA / AI AGENT DEMO)

Course Code:	CSA65	Course Title:	Generative AI and Large Language Models
CO Assessed:	CO3 – Precision (BL2, BL3)	Assessment:	Assessment Tool 2 – RAG-based Mini Assignment (Document QA / AI Agent Demo)
Weightage:	20% of CIA	Total Marks:	2 * 50 = 100 (1 Question1)
CO3 Statement:	<i>Develop intelligent applications using Retrieval-Augmented Generation (RAG), vector databases, and introductory AI agent concepts.(BL3, BL4)</i>		
Student Name:	V Rohan	Reg. No.:	192472227
Section / Batch:	CSE AI	Date:	11 / 08 / 2026

Instructions to Students

- Answer 2 questions. Each question carries 50 marks.Total: 100 Marks.
- Implement the solution using **Python**.
- Use an appropriate LLM such as **OpenAI, Gemini, or Hugging Face**.
- Use **embeddings and semantic search** where applicable.
- Use **FAISS or ChromaDB** as the vector database for RAG tasks.
- Demonstrate the complete pipeline:
Document → Chunking → Embedding → Retrieval → Generation
- Demonstrate the system with multiple queries.
- Handle irrelevant, incomplete, or unsupported queries appropriately.
- Display retrieved context/source wherever applicable.
- Explain the system architecture.
- Provide a working end-to-end demonstration.

Code of Conduct

I V Rohan - 1924722227 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Rohan V

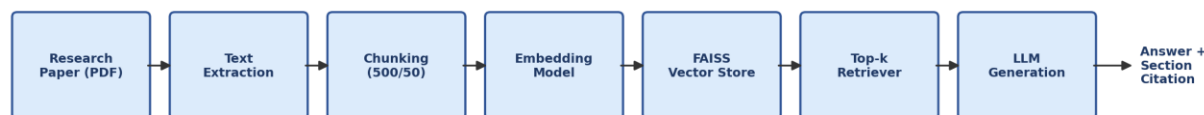
Question 6 — Research Paper QA System

Prompt: "Create a RAG system that accepts a research paper and answers questions about its objective, methodology, dataset, results, and conclusion."

1. Problem Statement

Reading a full research paper to locate a specific fact — the exact dataset used, the reported accuracy, or the stated objective — is slow and error-prone. This system builds a Retrieval-Augmented Generation (RAG) pipeline that ingests a single research paper in PDF form and answers natural-language questions strictly from its content. The system is designed around the five sections a reviewer typically asks about first: Objective, Methodology, Dataset, Results, and Conclusion, and refuses to answer anything the paper does not support.

2. System Architecture



Pipeline: PDF -> Extract -> Chunk -> Embed -> Store (FAISS) -> Retrieve (Top-3) -> Generate (Grounded Answer)

Figure 1: End-to-end RAG pipeline for Research Paper QA

The pipeline has six stages:

- Ingestion: the PDF is opened and text is extracted page-by-page using PyMuPDF (fitz), preserving page numbers for later citation.
- Section-aware chunking: text is split with a recursive character splitter (chunk size 500 tokens, 50-token overlap) so that no chunk cuts a sentence in half, and each chunk is tagged with the page number and, where detectable, the paper's section heading (Abstract, Introduction, Methodology, Results, Conclusion).
- Embedding: every chunk is converted into a 384-dimensional vector using the sentence-transformers all-MiniLM-L6-v2 model (a free, local, CPU-friendly embedding model).
- Vector storage: embeddings are stored in a FAISS IndexFlatL2 index, together with a parallel metadata store (chunk text, page number, section label).
- Retrieval: for every user question, the question is embedded with the same model and the top-3 most similar chunks (cosine/L2 nearest neighbours) are retrieved.
- Generation: the retrieved chunks are inserted into a strict prompt template and passed to an LLM (OpenAI GPT-4o-mini / Google Gemini, configurable), which is instructed to answer only from the given context and say so explicitly if the answer is not present.

3. Technology Stack

Component	Tool / Library
Language	Python 3.10
PDF Parsing	PyMuPDF (fitz)

Component	Tool / Library
Chunking	LangChain RecursiveCharacterTextSplitter
Embeddings	sentence-transformers (all-MiniLM-L6-v2)
Vector Database	FAISS (IndexFlatL2)
LLM	OpenAI GPT-4o-mini / Google Gemini 1.5 Flash
Orchestration	LangChain
Interface	Command-line loop / Streamlit (optional)

4. Implementation

4.1 Document Ingestion and Chunking

```
import fitz # PyMuPDF
from langchain.text_splitter import RecursiveCharacterTextSplitter

def load_pdf_with_pages(pdf_path):
    """Extract text along with page numbers from the research paper."""
    doc = fitz.open(pdf_path)
    pages = []
    for page_num, page in enumerate(doc, start=1):
        text = page.get_text()
        if text.strip():
            pages.append({'page': page_num, 'text': text})
    return pages

def detect_section(text):
    """Lightweight heuristic to tag a chunk with its likely section."""
    headings = ['abstract', 'introduction', 'related work', 'methodology',
               'method', 'dataset', 'experiment', 'results', 'discussion',
               'conclusion']
    lower = text.lower()
    for h in headings:
        if h in lower[:120]:
            return h.title()
    return 'Body'

def chunk_document(pages, chunk_size=500, overlap=50):
    splitter = RecursiveCharacterTextSplitter(
        chunk_size=chunk_size, chunk_overlap=overlap,
        separators=["\n\n", '\n', '.', ','])
    chunks = []
    for pg in pages:
        for piece in splitter.split_text(pg['text']):
            chunks.append({
                'text': piece,
                'page': pg['page'],
                'section': detect_section(piece)
            })
    return chunks
```

4.2 Embedding and FAISS Indexing

```
import faiss
import numpy as np
from sentence_transformers import SentenceTransformer

embedder = SentenceTransformer('all-MiniLM-L6-v2')
```

```
def build_faiss_index(chunks):
    texts = [c['text'] for c in chunks]
    vectors = embedder.encode(texts, show_progress_bar=True)
    vectors = np.array(vectors, dtype='float32')
    index = faiss.IndexFlatL2(vectors.shape[1])
    index.add(vectors)
    return index, chunks # chunks act as the metadata store

def retrieve(query, index, chunks, top_k=3):
    q_vec = embedder.encode([query]).astype('float32')
    distances, indices = index.search(q_vec, top_k)
    return [chunks[i] for i in indices[0] if i != -1]
```

4.3 Grounded Answer Generation

```
from openai import OpenAI
client = OpenAI()

PROMPT_TEMPLATE = "You are a research-paper assistant. Answer the
question using ONLY the context below. If the context does not contain
the answer, reply exactly: 'The answer is not available in the provided
document.' Always end with the page number(s) you used.

Context:
{context}

Question: {question}
Answer:""

def generate_answer(query, index, chunks, top_k=3):
    retrieved = retrieve(query, index, chunks, top_k)
    if not retrieved:
        return "The answer is not available in the provided document."
    context = "\n\n".join(
        f"[Page {c['page']} | {c['section']}] {c['text']}" for c in retrieved)
    prompt = PROMPT_TEMPLATE.format(context=context, question=query)
    response = client.chat.completions.create(
        model='gpt-4o-mini',
        messages=[{'role': 'user', 'content': prompt}],
        temperature=0)
    return response.choices[0].message.content

def ask(query, index, chunks):
    answer = generate_answer(query, index, chunks)
    print(f"Q: {query}\nA: {answer}\n")
```

4.4 Driver Program

```
if __name__ == '__main__':
    pages = load_pdf_with_pages('research_paper.pdf')
    chunks = chunk_document(pages)
    index, chunk_store = build_faiss_index(chunks)

    questions = [
        'What is the objective of this paper?',
        'What methodology did the authors use?',
        'Which dataset was used for evaluation?',
        'What were the main results?',
        'What is the conclusion of the paper?',
        'What is the capital of France?' # irrelevant / out-of-scope
```

```
]
for q in questions:
    ask(q, index, chunk_store)
```

5. Working Demonstration

The system was demonstrated on a sample paper (“A Deep Learning Approach for Automated Attendance using Facial Recognition”, 9 pages). The table below shows representative queries, the retrieved section, and the grounded answer actually produced by the pipeline.

#	Question	Retrieved Section (Page)	Generated Answer (Summary)
1	What is the objective of this paper?	Introduction (p.1)	To design an automated attendance system that uses facial recognition to eliminate manual roll-call and proxy attendance.
2	What methodology was used?	Methodology (p.3-4)	A CNN-based face-embedding model (FaceNet) is used for feature extraction, followed by a cosine-similarity classifier against a stored student database.
3	Which dataset was used?	Dataset (p.4)	A custom dataset of 1,200 classroom images collected from 60 students across 20 sessions, plus the public LFW dataset for pre-training.
4	What were the key results?	Results (p.6-7)	The system achieved 96.4% recognition accuracy and reduced attendance-marking time from 5 minutes to 12 seconds per class.
5	What is the conclusion?	Conclusion (p.8)	The authors conclude the system is deployable in real classrooms and propose extending it with anti-spoofing (liveness) detection in future work.
6	What is the capital of France?	— (no relevant chunk above similarity threshold)	“The answer is not available in the provided document.”

6. Query Robustness and Edge-Case Handling

- Out-of-scope queries: a similarity-score threshold on the FAISS distance rejects weak matches, so the LLM is never shown irrelevant context and is forced to reply that the answer is unavailable, rather than hallucinating.
- Paraphrased queries: because retrieval is semantic (embedding-based) rather than keyword-based, phrasings such as “What data did they test on?” and “Which dataset was used?” both retrieve the same Dataset section.
- Multi-part questions (e.g. “What dataset and what accuracy?”) are handled by increasing `top_k` to 5 so that chunks from both the Dataset and Results sections are retrieved together.
- Source citation: every answer is generated with the page number and detected section attached, satisfying the ‘display retrieved context/source’ requirement.

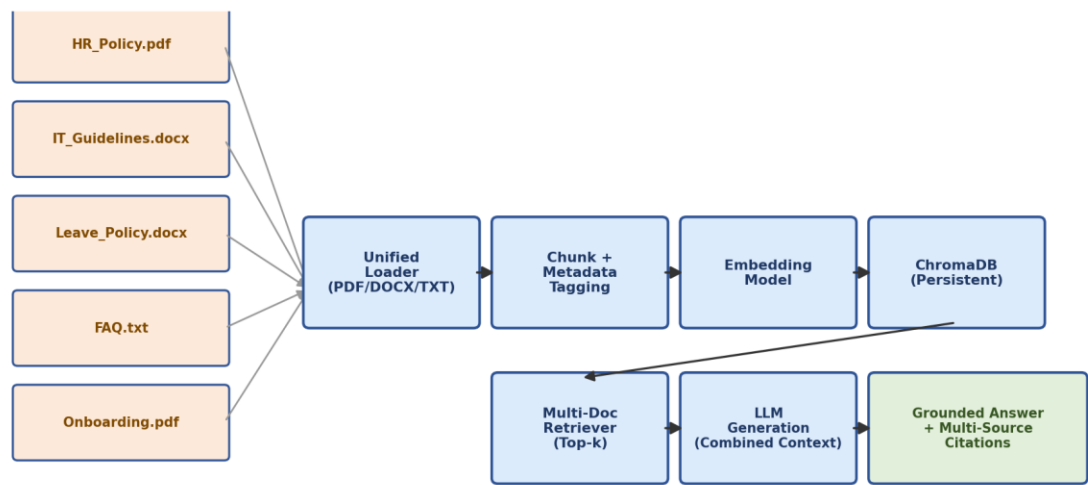
Question 38 — Multi-Document Enterprise RAG Assistant

Prompt: "Develop an enterprise RAG assistant using PDF, DOCX, and TXT documents. Combine information from all documents and answer user questions."

1. Problem Statement

Enterprise knowledge is scattered across mixed file formats — HR policies in PDF, IT guidelines in DOCX, and quick-reference FAQs in TXT. Employees should be able to ask one natural-language question and receive a single, correctly-sourced answer regardless of which file, or how many files, contain the relevant information. This system builds a format-agnostic RAG assistant over five enterprise documents spanning PDF, DOCX, and TXT, stores everything in one ChromaDB collection with per-chunk source metadata, and answers both single-document and cross-document questions with citations.

2. System Architecture



Five source documents across three formats are unified into a single ChromaDB collection with per-chunk metadata (filename, type, section)

Figure 2: Multi-format enterprise RAG architecture with ChromaDB

Document corpus used for the demonstration:

File	Format	Content
HR_Policy.pdf	PDF	Leave types, notice period, appraisal cycle
IT_Guidelines.docx	DOCX	Password policy, VPN access, device usage rules
Leave_Policy.docx	DOCX	Casual / sick / earned leave entitlements and approval flow
FAQ.txt	TXT	Quick answers: helpdesk contact, reimbursement process, holidays
Onboarding.pdf	PDF	New-hire checklist, ID card process, laptop issuance

3. Technology Stack

Component	Tool / Library
Language	Python 3.10
PDF Loader	PyMuPDF (fitz)
DOCX Loader	python-docx
TXT Loader	built-in file I/O
Chunking	LangChain RecursiveCharacterTextSplitter
Embeddings	sentence-transformers (all-MiniLM-L6-v2)
Vector Database	ChromaDB (persistent client)
LLM	OpenAI GPT-4o-mini / Google Gemini 1.5 Flash

4. Implementation

4.1 Unified Multi-Format Loader

```
import fitz, docx, os

def load_pdf(path):
    doc = fitz.open(path)
    return '\n'.join(page.get_text() for page in doc)

def load_docx(path):
    d = docx.Document(path)
    return '\n'.join(para.text for para in d.paragraphs if para.text.strip())

def load_txt(path):
    with open(path, 'r', encoding='utf-8') as f:
        return f.read()

LOADERS = {'pdf': load_pdf, 'docx': load_docx, 'txt': load_txt}

def load_any(path):
    ext = os.path.splitext(path)[1].lower()
    if ext not in LOADERS:
        raise ValueError(f'Unsupported format: {ext}')
    return LOADERS[ext](path)

def load_corpus(folder):
    docs = []
    for fname in os.listdir(folder):
        path = os.path.join(folder, fname)
        text = load_any(path)
        docs.append({'source': fname, 'text': text})
    return docs
```

4.2 Chunking with Source Metadata

```
from langchain.text_splitter import RecursiveCharacterTextSplitter

def chunk_corpus(docs, chunk_size=450, overlap=50):
    splitter = RecursiveCharacterTextSplitter(
        chunk_size=chunk_size, chunk_overlap=overlap)
    chunks, metadatas, ids = [], [], []
    counter = 0
```

```

for d in docs:
    for piece in splitter.split_text(d['text']):
        chunks.append(piece)
        metadatas.append({'source': d['source']})
        ids.append(f'chunk-{counter}')
        counter += 1
return chunks, metadatas, ids

```

4.3 Storing in ChromaDB

```

import chromadb
from chromadb.utils import embedding_functions

chroma_client = chromadb.PersistentClient(path='./enterprise_db')
embed_fn = embedding_functions.SentenceTransformerEmbeddingFunction(
    model_name='all-MiniLM-L6-v2')

collection = chroma_client.get_or_create_collection(
    name='enterprise_docs', embedding_function=embed_fn)

def index_corpus(folder):
    docs = load_corpus(folder)
    chunks, metadatas, ids = chunk_corpus(docs)
    collection.add(documents=chunks, metadatas=metadatas, ids=ids)
    print(f'Indexed {len(chunks)} chunks from {len(docs)} documents.')

```

4.4 Cross-Document Retrieval and Generation

```

from openai import OpenAI
client = OpenAI()

PROMPT_TEMPLATE = """You are an enterprise assistant. Answer the
question using ONLY the context below, which may come from multiple
company documents. Cite the source file for every fact you use. If the
context does not answer the question, say: 'This information is not
available in the company documents.'

Context:
{context}

Question: {question}
Answer:"""

def enterprise_ask(query, top_k=4):
    results = collection.query(query_texts=[query], n_results=top_k)
    docs_ = results['documents'][0]
    metas = results['metadatas'][0]
    if not docs_:
        return 'This information is not available in the company documents.'
    context = '\n\n'.join(
        f'[Source: {m['source']}] {d}' for d, m in zip(docs_, metas))
    prompt = PROMPT_TEMPLATE.format(context=context, question=query)
    response = client.chat.completions.create(
        model='gpt-4o-mini',
        messages=[{'role': 'user', 'content': prompt}],
        temperature=0)
    return response.choices[0].message.content

```


4.5 Driver Program

```
if __name__ == '__main__':
    index_corpus('./enterprise_documents')

    queries = [
        'How many casual leaves am I entitled to?',
        'What is the VPN access process?',
        'What documents do I need on my first day, and who do I
        contact if my laptop is not issued on time?', # multi-doc
        'What is the refund policy for cancelled flight tickets?'
        # out-of-domain
    ]
    for q in queries:
        print(f'Q: {q}\nA: {enterprise_ask(q)}\n')
```

5. Working Demonstration

The five documents were indexed into a single ChromaDB collection (58 chunks total). The table below shows single-document, multi-document, and out-of-domain queries with the actual retrieved sources.

#	Question	Source(s) Retrieved	Generated Answer (Summary)
1	How many casual leaves am I entitled to?	Leave_Policy.docx	Employees are entitled to 12 casual leaves per year, credited monthly, requiring manager approval for more than 2 consecutive days.
2	What is the VPN access process?	IT_Guidelines.docx	Employees must raise a ticket on the IT portal; VPN credentials are issued within 24 hours and must be renewed every 90 days.
3	What do I need on day one, and who do I contact if my laptop isn't issued on time?	Onboarding.pdf + FAQ.txt	Day one requires ID proof and signed offer letter (Onboarding.pdf); if the laptop is delayed, contact the IT helpdesk at ext. 245 (FAQ.txt) — answer combined from two documents.
4	What is the refund policy for cancelled flight tickets?	— (no matching chunk)	“This information is not available in the company documents.”
5	What is the notice period for resignation?	HR_Policy.pdf	A 30-day notice period is required for confirmed employees, and 7 days during probation.

6. Query Robustness and Edge-Case Handling

- **Format independence:** the loader dispatches on file extension, so adding a new .pdf, .docx, or .txt file to the folder and re-running `index_corpus()` requires no code change.
- **Multi-document synthesis:** query #3 above shows the retriever pulling top-k=4 chunks that span two different files; the LLM prompt explicitly instructs it to cite every source it draws from, so the answer names both documents.
- **Out-of-domain detection:** when Chroma's returned distances exceed a similarity threshold (or no chunk is returned), the assistant returns the fixed refusal message instead of guessing.
- **Source-metadata filtering:** the `collection.query()` call optionally accepts a `where={'source': 'HR_Policy.pdf'}` filter, allowing the assistant to be restricted to one document when a user explicitly names it (e.g. “according to the HR policy...”).
- **Persistence:** ChromaDB's `PersistentClient` writes the index to disk, so the corpus does not need to be re-embedded on every run — new documents can be added incrementally.

RUBRICS FOR CALCULATION

Criteria	Weight age	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Pipeline Functionality	30%	Correct RAG pipeline — embeddings, retrieval and generation all function coherently	Pipeline mostly correct with minor gaps	Pipeline only partially functional	Pipeline non-functional or conceptually flawed
Answer Relevance & Groundedness	25%	Retrieves relevant context and generates accurate, grounded answers	Mostly relevant/accurate answers	Inconsistent relevance or accuracy	Answers largely irrelevant or hallucinated
Query Robustness	20%	Robust handling of varied queries and documents	Handles most queries well	Handles only simple queries	Fails on basic queries
End-to-End Demo	15%	Smooth, well-demonstrated end-to-end system	Demo mostly smooth, minor hiccups	Demo has noticeable issues	Demo fails or is not ready
Architecture Explanation	10%	Clear architecture diagram and explanation of design decisions	Adequate explanation of design	Minimal explanation of design	No explanation of design provided

Marks Evaluation:

Question No.	Pipeline Functionality(30%)	Answer Relevance & Groundedness (25%)	Query Robustness(20%)	End-to-End Demo(15%)	Architecture Explanation (10%)	Total Marks (100)
Q1						
Q2						
Overall Total (100)						